Introduction

Fitz Koch

2026-06-12

Overview

This mini-website will serve as an introduction to the skills and projects we'll be working on during the summer of 2026 for the Okemo Verso Pod.

Skills

1. `Git` is really great for working on projects together as well as for storing your stuff cheaply. Hopefully VERSO's main onboarding helped with that.
2. Quarto Notebooks (`.qmd`) give you the advantages of jupyter notebooks in a more replicable and git-friendly way:
    - Incredibly versatile, and can include any `Python` code as well as `R`.
    - Interactive coding environment
    - One document sources converts to:
        - Interactive slides
        - `.html` for websites, just like this one
        - `.ipynb` jupyter notebook files
        - `.pdf` files. Note this can use Latex, like: $2 = \frac{10}{5}$
    - Are purely text-based, so git differentials make sense and are actually readable.
3. Coding practices
    - Using a project-wide formatter. We'll use `ruff`.
        - Keeps code readable and short
        - Minimizes unnecessary `git` differentials.
    - Proper `git` usage
        - commit on every meaningful change
        - pull request every complete feature.
        - respond to, and fix, reviews.

- Type hints
  - clearer code
  - easier bug hunts

4. Data Engineering for larger-scale data-science
   - ETL
   - See how data works when you have a lot of it that you want to use at different times, many times, in different ways.
5. Policy Research
   - Building good projects often means translating prior work or uncertain requests. This will help with that.
6. Typescript (Optional)
   - learn true power of types
   - learn how internet actually works

Projects

1. Benefits Cliff
   - Languages: `qmd` files (no code) and `typescript` logic
   - Goal: help legislators and everyday people understand:
     - what benefits are available
     - how those benefits change with income or family situation
     - how those benefits relate with census information about income and family situation
   - Work to be done:
     - Adding more benefits
       * Policy review: build out core logic in `qmd` files for calculating benefits from policy manuals and other resources
       * typescript: code up algorithms
       * backfill: work through existing typescript algorithms and backfill the logic into `qmd` documentation.
     - Part 2: Extension
       * Add in information about resources required for life per county
       * Data analysis: pull in census information, etc., and see how comparisons work out.
2. Vermont Reports
   - Languages: `SQL` (duckdb), `qmd`, and `Python`.

- Goal: help citizens, select boards, local government, and planners better understand their towns
- Work to be done:
  - Data Engineering:
    - Updating all data engineering logic :
      1. get data (external APIs or raw download)
      2. build clean data (SQL and python scripts)
         - requires lots of though and consultation about how we should most effectively split up the tables.
      3. runtime queries / logic (SQL and python scripts)
      4. api (pythons' FASTAPI library)
      5. frontend
    - Removing and cleaning legacy code
  - Analysis:
    - Adding in new functionality and places for analysis, especially putting different data streams in conversation with each other
      - EG, how broadband and wastewater infrastructure inter-relate.

3. TBD:
   - Languages: almost certainly `SQL, Python, .qmd`
   - Goal: help people of Vermont. Working with Kendall to see exactly what that is.

Next Steps

Complete the following tutorials:

1. Git: complete standard VERSO repository.
2. Complete the Data Engineering In Quarto project
3. Complete the Basic Typescript project (COMING SOON)

Work Projects:

1. Data Engineering in Vermont Data Collaborative